

♥ Like

📌 Save

🔗 Share



@thedevspaceio

JavaScript Objects Cheatsheet



Object methods and operations

Method	Description
<code>Object.keys()</code>	Returns an array of own enumerable property names.
<code>Object.values()</code>	Returns an array of own enumerable property values.
<code>Object.entries()</code>	Returns an array of <code>[key, value]</code> pairs.
<code>Object.fromEntries()</code>	Creates an object from <code>[key, value]</code> pairs.
<code>Object.assign()</code>	Copies properties from source objects to a target.
<code>Object.create()</code>	Creates a new object with a specified prototype.
<code>Object.defineProperty()</code>	Defines or modifies a property with a descriptor.
<code>Object.defineProperties()</code>	Defines or modifies multiple properties.
<code>Object.getOwnPropertyDescriptor()</code>	Returns the descriptor for a property.
<code>Object.getOwnPropertyNames()</code>	Returns all own property names.
<code>Object.getOwnPropertySymbols()</code>	Returns all own symbol properties.

Method	Description
<code>Object.freeze()</code>	Prevents modifications to an object.
<code>Object.seal()</code>	Prevents adding/removing properties.
<code>Object.preventExtensions()</code>	Prevents adding new properties.
<code>Object.isFrozen()</code>	Checks if an object is frozen.
<code>Object.isSealed()</code>	Checks if an object is sealed.
<code>Object.isExtensible()</code>	Checks if new properties can be added.
<code>Object.hasOwn()</code>	Checks if a property is an own property.
<code>Object.getPrototypeOf()</code>	Returns the prototype of an object.
<code>Object.setPrototypeOf()</code>	Sets the prototype of an object.
<code>Object.is()</code>	Compares two values, including <code>NaN</code> and <code>-0</code> .

Creating objects

Object literal

js



```
1  const user = {  
2    name: "Alice",  
3    age: 30,  
4  };
```

Object.create()

Creates an object with a specified prototype.

js



```
1  const proto = {  
2    greet() {  
3      return "hello";  
4    },  
5  };  
6  const obj = Object.create(proto);  
7  obj.greet(); // "hello"  
8  
9  const dict = Object.create(null); // no prototype
```

Accessing properties

Dot and bracket notation

js



```
1 const o = { a: 1, "b-key": 2 };
2 o.a; // 1
3 o["b-key"]; // 2
```

Computed property names

js



```
1 const key = "name";
2 const user = { [key]: "Alice" };
3 user.name; // "Alice"
```

Iterating objects

`Object.keys()`

Returns an array of property names.

js



```
1 const o = { a: 1, b: 2 };  
2 Object.keys(o); // ["a", "b"]
```

`Object.values()`

Returns an array of property values.

js



```
1 Object.values(o); // [1, 2]
```

`Object.entries()`

Returns an array of `[key, value]` pairs.

js



```
1 Object.entries(o); // [{"a", 1}, {"b", 2}]
```

`Object.fromEntries()`

Creates an object from `[key, value]` pairs.

js



```
1 const entries = [  
2   ["a", 1],  
3   ["b", 2],  
4 ];  
5 Object.fromEntries(entries); // { a: 1, b: 2 }
```

Copying and merging

Spread syntax

js



```
1 const o = { a: 1, b: 2 };
2 const copy = { ...o, c: 3 }; // { a: 1, b: 2, c: 3 }
```

Object.assign()

Copies properties from source objects to a target.

js



```
1 const target = { a: 1 };
2 Object.assign(target, { b: 2 }, { c: 3 });
3 // target is { a: 1, b: 2, c: 3 }
```


Property descriptors

`Object.defineProperty()`

Defines or modifies a property with a descriptor.

js



```
1  const obj = {};  
2  Object.defineProperty(obj, "name", {  
3    value: "Alice",  
4    writable: false,  
5    enumerable: true,  
6    configurable: false,  
7  });  
8  obj.name = "Bob"; // ignored in strict mode
```

`Object.getOwnPropertyDescriptor()`

Returns the descriptor for a property.

js



```
1  Object.getOwnPropertyDescriptor(obj, "name");  
2  // { value: "Alice", writable: false, enumerable: true, configurab
```

Immutability helpers

`Object.freeze()`

Prevents adding, removing, or changing properties. Shallow freeze.

js



```
1 const o = { a: 1 };
2 Object.freeze(o);
3 o.a = 2; // ignored in strict mode
```

`Object.seal()`

Prevents adding or removing properties, but values can still be changed.

js



```
1 const o = { a: 1 };
2 Object.seal(o);
3 o.a = 2; // works
4 o.b = 3; // ignored
```

`Object.preventExtensions()`

Prevents adding new properties.

js



```
1  const o = { a: 1 };
2  Object.preventExtensions(o);
3  o.b = 2; // ignored
```

Checking properties

`Object.hasOwn()`

Checks if a property is an own property (not inherited).

js



```
1 const o = { a: 1 };
2 Object.hasOwn(o, "a"); // true
3 Object.hasOwn(o, "toString"); // false
```

`in` operator

Checks if a property exists anywhere in the prototype chain.

js



```
1 const user = { name: "Alice" };
2
3 "name" in user; // true (own property)
4 "toString" in user; // true (inherited from Object.prototype)
5 "age" in user; // false
6
7 // Compare with Object.hasOwn()
8 Object.hasOwn(user, "name"); // true
9 Object.hasOwn(user, "toString"); // false
```

Use `in` when you want to check inherited properties, and `Object.hasOwn()` when you only want own properties.

Prototypes

`Object.getPrototypeOf()`

Returns the prototype of an object.

js



```
1  const obj = {};  
2  Object.getPrototypeOf(obj) === Object.prototype; // true  
3  
4  const arr = [];  
5  Object.getPrototypeOf(arr) === Array.prototype; // true
```

`Object.setPrototypeOf()`

Changes the prototype of an object. Use carefully because it can hurt performance.

js



```
1  const animal = {  
2    speak() {  
3      return "sound";  
4    },  
5  };  
6  const dog = { breed: "lab" };  
7  
8  Object.setPrototypeOf(dog, animal);  
9  dog.speak(); // "sound"  
10 Object.getPrototypeOf(dog) === animal; // true
```